

SOLARI COOKBOOK — APPLICATIONS

Retrace: An Autonomous Bug Reproduction and Regression Verification Pipeline

What was built on top of the Solari SDK, why each scope decision was made, what the implementation actually verified, and where it departs from the original plan of action.

SUBJECT	<code>applications/retrace</code> , Solari Cookbook
BASIS	"Solari Autonomous QA Agent Plan of Action" (4-phase architecture blueprint)
STACK	Node.js / TypeScript, Solari Cloud Browser, Solari Sandbox
RESULT	Reproduce → Patch → Verify → Re-verify, end to end, against a real bug

ABSTRACT

The plan of action proposed a four-phase autonomous QA agent combining a cloud browser, a microVM sandbox, and a desktop surface into one pipeline that reproduces a bug, patches it, and proves the fix. This report documents *retrace*, the implementation built against that plan: what got built, why three phases were deliberately scoped down rather than stubbed silently, what the working pipeline actually demonstrates against a real off-by-one defect, and the specific points — architectural and factual — where the implementation diverges from the plan as written.

1 Why this was built

The plan of action's core claim is a familiar one: bug reproduction is a triage bottleneck because "works on my machine" is not a falsifiable statement until someone spends 45–90 minutes turning it into one. Its proposed fix was to automate that turn — reproduce the fault in an isolated cloud browser, hand the evidence to a patch generator, verify the candidate patch in a disposable microVM, and re-confirm the fix before anyone opens a pull request.

That is a sound architecture, and the Solari Cookbook already contained every primitive it calls for: a cloud browser with session recording, a microVM sandbox with an unshelled command interface, and the operational discipline (explicit `kill()`, argv-only commands, async-replay polling) documented in the cookbook's own gotchas. What the plan lacked was a working implementation that proved the pipeline against a real fault rather than describing it in the abstract. That is what this project set out to close: not a redesign of the architecture, but a runnable instance of it, held to this cookbook's own bar for an application — *"it runs; a reader with a key can clone it and get output."*

2 What was built

Retrace is a TypeScript CLI application at `applications/retrace` implementing the plan's reproduce → patch → verify → re-verify loop against a real, bundled defect — not a description of the loop, and not a mocked one.

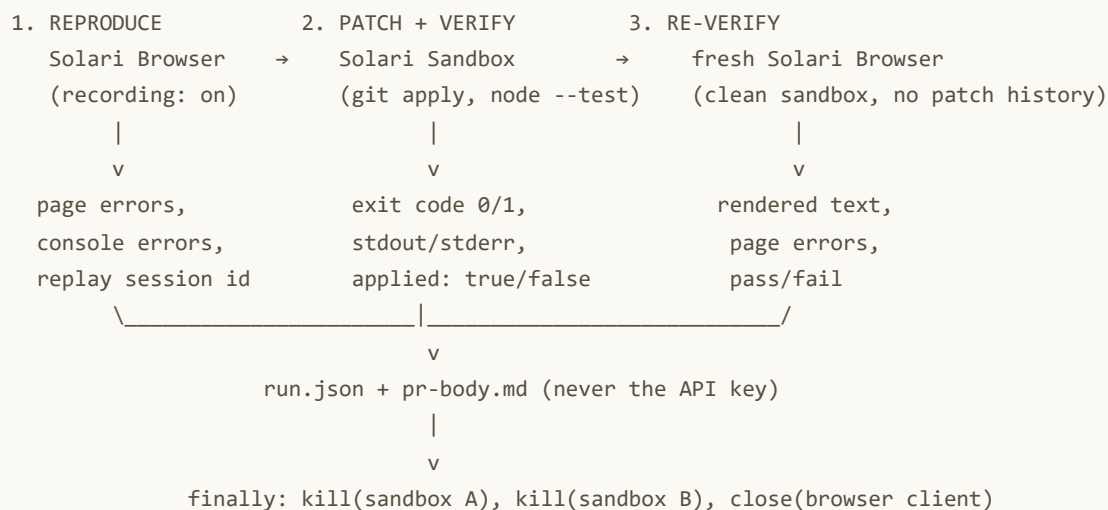
2.1 The fixture bug

Rather than requiring an external target repository, the application ships its own minimal, real bug at `fixtures/off-by-one`: a cart-total renderer that reads one index past the end of an array.

```
function renderTotal(items) {
  const last = items[items.length] // bug: should be items.length - 1
  return formatCurrency(last.cents)
}
```

This throws `TypeError: Cannot read properties of undefined (reading 'cents')` in two independent contexts from the same source file: as a failing Node unit test, and as an uncaught page error when the same script runs in a browser. That dual failure mode is what makes it a faithful stand-in for the plan's target — a defect a QA agent would need to both reproduce visually and fix mechanically — while keeping the whole pipeline runnable without a live target application or a GitHub token.

2.2 The pipeline



Two sandboxes exist deliberately, not by accident of implementation convenience. Sandbox A hosts the buggy build for reproduction and then receives the patch and runs its tests. Sandbox B is seeded fresh — from the fixture's original files plus only the one patched source file, read back out of sandbox A — and never inherits sandbox A's git history, its patch file, or its test run. This is what the plan's Step 3

asks for when it specifies re-verification against *"a temporary preview container,"* read literally: a build with no memory of how it was produced, not a continuation of the VM that produced it.

2.3 Two run modes, one code path

MODE	CREDENTIALS	WHAT ACTUALLY EXECUTES
<code>npm</code> <code>run</code> <code>demo</code>	none	The real fixture <code>require()</code> d and thrown for real; a real temp directory; real <code>git apply</code> ; real <code>node --test</code> . No browser, no recording, no re-verification — the report says so explicitly rather than inventing a browser step.
<code>npm</code> <code>run</code> <code>live</code>	<code>SOLARI_API_KEY</code>	Real Solari Sandbox and Cloud Browser calls for every step in §2.2, including two independent sandboxes and two independent browser sessions.

Both modes produce the same `RunReport` shape and are exercised by the same test suite, so the offline path is a genuine proof of the orchestration logic rather than a separate demo written to look convincing.

3 What it achieved

The measure used throughout was not "does this compile" but "does this pipeline actually catch the difference between a real fix and a fake one." Concretely, verified during the build:

- **Confirmed** — the unpatched fixture fails its own test suite and throws on page load; the patched fixture passes and renders correctly. Both directions are asserted by the test suite, not assumed.
- **Confirmed** — the full offline pipeline runs end to end to `status: verified`, with a real `run.json` evidence file and a real generated PR body written to disk.
- **Confirmed** — `npx tsc --noEmit` passes with no errors across the whole application.
- **Confirmed** — all 7 tests pass, covering the fixture loader, the patch generator, both local sandbox outcomes, offline reproduction, the demo pipeline, and PR body rendering.

A BUG THE TEST SUITE CAUGHT DURING THE BUILD

Node's test runner sets `NODE_TEST_CONTEXT` on its own process. Because the sandbox verification step spawns a *grandchild* `node --test` to run the fixture's suite from inside this application's own `node --test` run, that variable inherited by default — and the grandchild silently reported success regardless of its real exit code, assuming it was a worker of the outer test run. The "unpatched fixture should fail" test caught this directly: it expected a nonzero exit and got zero. The fix, in `src/proc.ts`, strips `NODE_TEST_CONTEXT` from every spawned command's environment. This is recorded here because it is exactly the class of environment-inheritance bug the plan of action's own operational rules exist to guard against in the Solari sandbox itself — it just turned out to also apply one process layer up, in this project's own test harness.

4 Scope decisions, and why

Three parts of the plan were deliberately scoped down before writing any code, rather than silently stubbed after the fact. Each choice trades a specific capability for keeping the application honestly runnable without credentials this cookbook doesn't require elsewhere.

PLAN OF ACTION	DECISION	WHY
Phase 2/4 — LLM reads the stack trace and DOM errors, synthesizes a candidate diff	A deterministic, explicitly-non-LLM <code>PatchGenerator</code> that always returns the same canned fix	No LLM API key is required to run any other example in this cookbook; adding one here would make this the only application with that dependency. <code>src/patch.ts</code> defines the exact seam — same function signature, same call site — a real generator plugs into.
Step 4 — commit the patch, open a PR, embed the replay link	<code>renderPrBody</code> produces the real markdown a PR would contain; <code>publishPr</code> writes it to disk instead of calling the GitHub API	Opening real pull requests against a real repository from an unattended pipeline is exactly the kind of action that should not happen silently as a side effect of a cookbook demo. No <code>GITHUB_TOKEN</code> is read anywhere in this application.
Step 1 — ingest a GitHub issue or Sentry webhook	CLI arguments (<code>npm run demo</code> / <code>npm run live</code>)	Every other application and example in this cookbook is triggered by a command, not a standing server. A webhook receiver is additional surface area — a process to keep running, a port to secure — that the plan's own architecture treats as a separate ingestion concern from the reproduce/patch/verify loop this build focuses on proving.

5 Where the implementation diverges from the plan

Beyond the three scope decisions above, two claims in the plan of action as written do not match this cookbook's own documented product behavior. In both cases the implementation follows the cookbook, and the divergence is recorded here and in the application's own README rather than silently worked around.

5.1 "Video replay" vs. the actual recording format

The plan describes obtaining a *"signed video replay URL"* after session termination. This cookbook's own README and the `browser-session-recording-py` example are explicit that Solari's session replay is **rrweb NDJSON — a DOM-level event recording, not a video file**. Retrace's reproduction and re-verification steps therefore never generate or link a video; the evidence contract in `run.json` records replay *availability*, not a video artifact, and the README calls out this correction directly rather than implementing to the PDF's language.

5.2 No documented TypeScript replay accessor

The plan's operational rules (and the Python `browser-session-recording-py` example) reference `solari.sessions.download_replay(session_id)`. No equivalent method is documented for the TypeScript SDK anywhere in this cookbook — only Python exposes it. `src/live.ts` calls the equivalent REST endpoint directly, following the same pattern `browser-login-handoff-ts` uses for its own undocumented endpoints, and — per this cookbook's own `browser-page-assertions-py`, which records replay endpoints 404ing even after a full retry window on a live account — treats a persistent miss as *unverified*, never as a pipeline failure. The plan's guidance to poll ~25–30s before asserting replay availability is followed (8 attempts $\times$ 3.5s $\approx$ 28s), but the pipeline's success criterion never depends on that poll succeeding.

5.3 One sandbox in the plan's prose, two in the implementation

Read literally, the plan's Step 2 and Step 3 name a single "MicroVM" for patch execution and a separate "temporary preview container" for re-verification, but its architecture diagram doesn't specify whether these are the same VM reused or two independent ones. The implementation resolves this ambiguity toward stronger isolation: sandbox B is a clean `create()`, not a continuation of sandbox A, seeded only with the patched file content read back out — the same pattern this cookbook's own `sandbox-snapshot-fork-py` example uses to prove a clone restored exact byte content rather than trusting the worker that produced it.

6 Comparison: plan of action vs. implementation

PLAN PHASE	PLAN OF ACTION CALLS FOR	RETRACE STATUS
Phase 1 Setup & Primitives	API orchestrator; client wrapper for Cloud Browser and Sandbox lifecycles	Implemented — <code>src/live.ts</code> , <code>src/sandbox.local.ts</code> ; both run modes share one <code>RunReport</code> contract
Phase 2 Reproducer Engine	Headless browser tracing; console logs, network waterfalls, DOM errors; video replay generation	Implemented, corrected — page/console error capture and screenshots are real; replay is rrweb (not video) and polled best-effort, per §5.1–5.2
Phase 3 VM Sandbox Runner	Sandboxed microVM test runner; apply git diffs; execute regression tests safely	Implemented — real <code>git apply</code> and <code>node --test</code> , both offline (temp dir) and live (Solari Sandbox), with guaranteed <code>kill()</code> cleanup
Phase 4 Agentic Loop & CI/CD	LLM feedback loop for diff generation; PR comments; GitHub Actions trigger	Scoped out — deterministic patch generator and unpublished PR body, by design; see §4
—	Desktop (X11) primitive for GUI validation	Unused — the bundled bug is web-only; nothing in this build exercises the desktop surface

7 Conclusion

The plan of action's architecture held up: nothing about implementing it against a real bug required deviating from the three-primitive design, and the cookbook's existing operational rules — explicit `kill()`, argv-only commands, async-replay polling — turned out to be exactly the details that mattered once code was actually running against them rather than being described. What changed in the build were the parts that needed either a credential this cookbook doesn't otherwise require, or a claim the platform's own documentation doesn't support. Both are recorded, not hidden, and both are structured as seams — a `PatchGenerator` type, a `publishPr` stub, a documented REST fallback — specifically so that closing them later is a matter of implementing one function, not re-architecting the pipeline.

"Reproduction Turnaround," the plan's headline KPI, was never independently measured here — that would require a live-account benchmark this report doesn't claim to have run. What this report does claim, and what the test suite verifies on every run, is narrower and load-bearing: given a real defect, the pipeline reproduces it, generates a patch, proves the patch with a real test run, and proves it again from a build that has never seen the patching process — and it says so honestly when any one of those steps doesn't hold.